MACHINE LEARNING · LAB 1 OF 10 · STUDENT HANDOUT

First Contact with Data: NumPy, Pandas and an Honest First Model

Chapter 1 — Introduction to Machine Learning

Module	Machine Learning
Chapter	1 — Introduction to Machine Learning
Level	3rd year, Software Engineering engineering programme
Institution	ISIMM — Institut Supérieur d'Informatique et de Mathématiques de Monastir
Instructor	Professor Nizar Omheni
Duration	90 minutes
Format	Pairs (one notebook per pair)

1. Scenario and practical objective

Scenario (hypothetical)

You have just joined the small data team of a botanical garden. Volunteers measure iris flowers with a caliper (sepal and petal length and width, in cm), and the garden would like a first automatic suggestion of the species. Before anything sophisticated, your lead engineer asks you to:

1. prove that your machine is ready (reproducible environment);
2. explore the measurements and say *which* of them carry the signal;
3. deliver a first model **evaluated honestly** — on unseen data and against a trivial baseline;
4. review a teammate's script that claims **100 % accuracy**.

Practical objective. By the end of the session your pair submits one executed notebook containing a correct evaluation of a first classifier, its interpretation, a reusable evaluation function applied to a second dataset, and a written diagnosis of a flawed evaluation script.

2. Learning outcomes

#	By the end of the lab you can…	Task(s)	Evidence in your notebook
LO1	use NumPy vectorized operations, broadcasting and axis reductions, **predicting result shapes** before running	T2	shape-prediction table, standardized matrix, Q2
LO2	explore a tabular dataset with Pandas and Matplotlib and **argue with numbers** which features separate the classes	T3, T4	summary tables, 3-panel figure, Q3–Q4
LO3	implement and interpret an **honest evaluation**: stratified split, majority baseline, test accuracy, confusion matrix	T5, T6	printed metrics, confusion matrix, `honest_evaluation()`, Q5–Q6
LO4	**diagnose and correct** flawed evaluation code	T7	written diagnosis of two flaws, corrected code and numbers, Q7

3. Prerequisites and resources

Prerequisites. Python basics (variables, functions, loops, lists and dicts); Chapter 1 vocabulary: features, target (label), supervised learning, training/test data, baseline, generalization.

Resources supplied (keep them in one folder): `Lab01_Student_Handout.pdf` (this document), `lab01_starter.ipynb`, `check_env.py`, `buggy_pipeline.py`, `requirements.txt`, `README.md`.

Software. Python 3.10 or newer, `numpy`, `pandas`, `matplotlib`, `scikit-learn`, Jupyter (or VS Code with the Jupyter extension). No GPU, no paid service, no API key. Datasets (Iris, Wine) are built into scikit-learn — no download.

Prerequisite check — 3 minutes, on paper, at the start of the session

1. In the iris problem, what are the *features* and what is the *target*?
2. Is this task supervised or unsupervised? Why?
3. Why must a model not be evaluated on the data it was trained on?
4. What does a *majority-class baseline* predict for every new sample?
5. What is the shape of `np.zeros((3, 4)).mean(axis=0)`?

4. Setup

```
python -m venv ml-env
# Linux/macOS:  source ml-env/bin/activate      Windows:  ml-env\Scripts\activate
pip install -r requirements.txt
python check_env.py
jupyter notebook lab01_starter.ipynb
```

`check_env.py` must end with *"Your environment is ready for Lab 1."* If it does not, call the instructor **immediately**; if your machine cannot be fixed within 5 minutes, work on your partner's machine.

5. Schedule

Phase	Content	Duration
Introduction	Scenario, prerequisite check, quick correction	10 min
Setup and guided work	T1 Environment (5) · T2 NumPy (10)	15 min
Core work	T3 Pandas (8) · T4 Matplotlib (7) · T5 First honest model (20)	35 min
Transfer and error analysis	T6 Wine transfer (10) · T7 Code review (12)	22 min
Synthesis	Checklist, exit ticket, debrief	8 min
Total		**90 min**

Durations are planning estimates. If you are behind schedule at minute 60, go straight to **Task 7** and finish Task 6 after the session.

6. Tasks

Each task below lists its purpose, inputs, instructions, deliverable and the criteria you can use to check your own work. Write all ✎ answers in the notebook cells provided. **Hints** are in Appendix A — use them one level at a time.

Task 1 — Environment check *(5 min)*

- **Purpose.** Make your setup reproducible and verifiable.
- **Inputs.** `check_env.py`, first cells of the notebook.
- **Instructions.** Run `python check_env.py` in the terminal. In the notebook, run the setup cell and the version cell. Compare the two lists of versions.
- **Deliverable.** The executed version cell.
- **Self-check.** Versions printed in the notebook = versions printed by the script (otherwise the kernel is not using your venv).

Task 2 — NumPy: vectorization, broadcasting, axes *(10 min)*

- **Purpose.** Manipulate arrays without loops and reason about shapes — the daily language of every later lab.
- **Inputs.** Notebook section 2; the synthetic matrix `X_small` (created for this exercise).
- **Instructions.**
  1. Run cell 2.1 and observe vectorized arithmetic, scalar broadcasting, slicing and a boolean mask.
  2. **Before running**, fill the shape-prediction table (2.2) for `M.mean(axis=0)`, `M.mean(axis=1)`, `M + v`, `M + w`, `M + np.array([1, 2, 3])`. Then complete the code, run it and fill the *Actual* column.
  3. (2.3) Standardize the columns of `X_small` with **one** vectorized expression: $Z = (X - \mu)/\sigma$, where μ and σ are column means and standard deviations. Verify that each column of Z has mean ≈ 0 and standard deviation ≈ 1.
  4. Answer Q2.
- **Deliverable.** Completed table (predicted *and* actual), `Z` with its check, Q2.
- **Self-check.** You can explain, using the broadcasting rule, why exactly one of the five expressions raises an error. No `for` loop in 2.3.

Task 3 — Exploring the data with Pandas *(8 min)*

- **Purpose.** Know your data before modelling it.
- **Inputs.** `load_iris(as_frame=True)`.
- **Instructions.** Display `head()`, `shape` and `info()`; add a readable `species` column; count samples per species; compute `describe()` on the four features; compute the mean of each feature **per species** with `groupby`. Answer Q3.
- **Deliverable.** The tables above and Q3.
- **Self-check.** Your Q3 answer quotes numbers and compares the differences between species means with the global standard deviation (not raw means alone).

Task 4 — Seeing the structure with Matplotlib *(7 min)*

- **Purpose.** Turn the statistics of Task 3 into a visual argument, and make a prediction you will test in Task 5.
- **Instructions.** One figure, three panels: (1) petal length vs petal width and (2) sepal length vs sepal width, one colour per species with a legend; (3) overlaid histograms of petal length per species (`alpha=0.5`). Label all axes. Answer Q4.
- **Deliverable.** The figure and Q4 — including an explicit **prediction** of where the classifier will make mistakes.
- **Self-check.** A reader can tell the species apart from your figure alone (legend, axis labels with units).

Task 5 — A first honest model *(20 min — core task)*

- **Purpose.** Apply the Chapter 1 evaluation discipline end to end.
- **Inputs.** `X` (4 features) and `y` (species code) from the DataFrame; `SEED = 42`.
- **Instructions.**

1. **Split**: `train_test_split` with `test_size=0.3, random_state=SEED, stratify=y`. Print shapes and class counts (`np.bincount`) for train and test.
2. **Baseline**: `DummyClassifier(strategy="most_frequent")`, fitted on train, accuracy on test.
3. **Model**: `KNeighborsClassifier(n_neighbors=3)` — it predicts the majority class among the 3 closest training points (Euclidean distance); you will study it in detail later. Fit on train; report accuracy on **test**; print train accuracy only for comparison; print the improvement over the baseline in percentage points.
4. **Errors**: show the confusion matrix with species names (rows = true, columns = predicted) and list the misclassified test samples. If time allows, mark them on the petal scatter plot.
5. Answer Q5 (a)–(e).

- **Deliverable.** Printed class counts, both test accuracies, confusion matrix, table of misclassified samples, Q5.
- **Self-check.** The baseline and the model are evaluated on the **same** test set; the number you call "the result" is a **test** accuracy; you have confronted the observed errors with your Q4 prediction.

Task 6 — Transfer: package and reuse the protocol *(10 min)*

- **Purpose.** Write the evaluation once as a function and test whether your conclusions carry over to a different dataset.
- **Inputs.** `load_wine(as_frame=True)` — 178 wines, 13 chemical measurements, 3 cultivars.
- **Instructions.** Complete `honest_evaluation(X, y, k=3, test_size=0.3, seed=SEED)` so that it returns the test accuracies of the baseline and of kNN in a dictionary. Check that it reproduces your Iris numbers from Task 5, then apply it **unchanged** to Wine. Print the Wine class counts and the min/max of `hue`, `magnesium` and `proline`. Answer Q6.
- **Deliverable.** The function, both results, Q6 with **one** testable hypothesis.
- **Self-check.** Calling your function on Iris gives exactly the Task 5 numbers. Your Q6 hypothesis refers to the min/max values you printed.

Task 7 — Code review: too good to be true *(12 min)*

- **Purpose.** Recognize the two most common evaluation mistakes when they appear in someone else's code.
- **Inputs.** `buggy_pipeline.py` (do not modify the model).
- **Instructions.** Run the script from the notebook (`!python buggy_pipeline.py`) and read it line by line. The script contains **two distinct evaluation flaws**. For each one: name it, point to the responsible line, and explain why the reported number is meaningless. Then write a corrected evaluation in the notebook and print the corrected numbers. Answer Q7.
- **Deliverable.** Q7 (two flaws) and the executed corrected code.
- **Self-check.** Your corrected numbers match your Task 5 numbers. You can explain why a "62-point improvement" in Report B does not prove anything, even if a correct evaluation might give a similar figure.

7. Deliverables and submission

Submit **one notebook per pair**, named `lab01_<name1>_<name2>.ipynb`, containing:

- all code cells completed and **executed in order** (*Kernel → Restart & Run All* before saving, no error);
- the shape-prediction table of Task 2 with predicted and actual columns;
- the 3-panel figure of Task 4;
- every ✎ answer (Q2–Q7, S1) and the exit ticket.

Submit through the channel indicated by your instructor, by the deadline announced in class.

8. Self-check checklist

- ☐ `check_env.py` and the notebook report the same package versions.
- ☐ I predicted every shape **before** running Task 2.
- ☐ My figure has a legend and labelled axes with units.
- ☐ My split is shuffled, stratified and seeded; I printed the class counts of train and test.
- ☐ Baseline and kNN are evaluated on the same **test** set; I report test accuracy, not train accuracy.
- ☐ I looked at *which* samples are misclassified, not only at the accuracy.
- ☐ `honest_evaluation()` reproduces my Task 5 numbers.
- ☐ I identified two distinct flaws in `buggy_pipeline.py` and my corrected numbers match Task 5.
- ☐ The notebook runs from top to bottom without error after a kernel restart.

9. Synthesis, reflection and exit ticket

✎ **S1 — Your evaluation checklist.** In at most four sentences, write the checklist you will apply to every model in this course.

Reflection. Which result surprised you most today, and what did it teach you about trusting a single accuracy number?

Exit ticket (last 3 minutes)

1. Why is a test accuracy meaningless if it is reported without a baseline?
2. A colleague announces "99 % accuracy". Which two questions do you ask first?
3. One thing from today that is still unclear to you.

10. Optional challenges

Not required for the learning outcomes — for pairs who finish early or want to go further at home.

- **C1.** Re-implement the majority-class baseline with NumPy only (`np.unique(..., return_counts=True)`) and check it matches `DummyClassifier`.
- **C2.** For $k = 1, \dots, 20$, compute the **5-fold cross-validation** accuracy **on the training set** (`sklearn.model_selection.cross_val_score`), plot it, choose k, and only then evaluate **once** on the test set. Explain why k must not be chosen by looking at test accuracy.
- **C3.** Put a `StandardScaler` before kNN on Wine with `make_pipeline(...)`. What happens to the accuracy? Does it support your Q6 hypothesis?
- **C4.** Time `sum(v * v for v in data)` against `np.dot(data, data)` on one million random numbers using `time.perf_counter()`.

Appendix A — Hint ladder

Read one hint at a time; try again before reading the next one.

Task 2

H1. Broadcasting compares shapes starting from the **last** dimension. **H2.** Two dimensions are compatible if they are equal or if one of them is 1. `(3, 4)` vs `(4,)`: compare 4 with 4. `(3, 4)` vs `(3,)`: compare 4 with 3. **H3.** For 2.3: `X_small.mean(axis=0)` has shape `(2,)` — one value per column — so `X_small - X_small.mean(axis=0)` subtracts it from every row.

Task 3

H1. `iris.target_names` is an array `['setosa', 'versicolor', 'virginica']`. **H2.** `dict(enumerate(iris.target_names))` gives `{0: 'setosa', ...}`; pass it to `df["target"].map(...)`. **H3.** `df.groupby("species")[features].mean()`.

Task 4

H1. `fig, axes = plt.subplots(1, 3, figsize=(15, 4))` gives three axes: draw on `axes[0]`, `axes[1]`, `axes[2]`. **H2.** Loop `for name, group in df.groupby("species"):` and call `ax.scatter(group[fx], group[fy], label=name)` once per species, then `ax.legend()`. **H3.** For the histogram, call `axes[2].hist(group["petal length (cm)"], alpha=0.5, label=name)` inside the same kind of loop.

Task 5

H1. `train_test_split` returns four arrays in the order `X_train, X_test, y_train, y_test`. **H2.** Every estimator follows the same pattern: `model.fit(X_train, y_train)`, then `model.predict(X_test)` or `model.score(X_test, y_test)`. **H3.** Misclassified positions: `np.where(y_pred != y_test)[0]`; species names: `iris.target_names[...]`.

Task 6

H1. Move the code of Task 5 inside the function and replace the global names by the parameters. **H2.** Use `.score(X_te, y_te)` for both models and return `{"baseline": ..., "knn": ...}`. **H3.** For Q6, look at how far apart the values of `proline` are compared with `hue`, and remember that kNN uses distances.

Task 7

H1. For each report, ask: *on which data was the model fitted, and on which data was it scored?* **H2.** Uncomment the two `bincount` lines at the end of the script. How many classes does the test set contain? **H3.** Look at the order of `y` in the original data (`print(y)`), then at the `shuffle` argument.

Appendix B — Reminders you need for this lab

- **Feature / target.** Features are the measured inputs (columns of `X`); the target is the value to predict (`y`).
- **Train / test split.** The model learns only from the training set; the test set simulates unseen data and is used to estimate generalization.
- **Stratification.** A stratified split keeps the class proportions of `y` identical in train and test.
- **Baseline.** The score of a trivial model on the same split. A *majority-class* baseline always predicts the most frequent training class; its accuracy equals that class's share of the test set.
- **Accuracy.** Fraction of correct predictions. **Confusion matrix.** Entry (i, j) counts samples of true class i predicted as class j; the diagonal holds correct predictions.

Appendix C — Troubleshooting

Symptom	Likely cause and fix
`ModuleNotFoundError` in the notebook but not in the terminal	The kernel is not your venv: run `pip install ipykernel` then `python -m ipykernel install --user --name ml-env` inside the venv, and select that kernel.
`pip` not found / wrong Python	Use `python -m pip install -r requirements.txt` with the venv activated.
`!python buggy_pipeline.py` says *file not found*	The notebook and the script are not in the same folder.
`ValueError: operands could not be broadcast together`	Expected in Task 2.2 — explain it in Q2. Elsewhere, print the shapes of both operands.
Plots do not appear	Make sure the cell ends with `plt.show()`.

Symptom	Likely cause and fix
Numbers differ slightly from your neighbours'	Check that you used `random_state=SEED` and `stratify=y` exactly as instructed.

Tested with Python 3.13.16, NumPy 2.5.3, pandas 3.0.5, Matplotlib 3.11.2, scikit-learn 1.9.1. The APIs used are long-standing, but if a call behaves differently with your versions, check the official documentation of your installed version.